Name: Priyanshi Singh
Class: T22
Roll no: 2401115

Experiment No. 02

Aim: To containerize applications using Docker.

LO Mapped: LO4

LO: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Theory:

1. Introduction to Containerization in DevOps

In modern Agile software development and DevOps environments, maintaining the same application behavior across different environments—such as development, testing, staging, and production—is a major challenge. This is commonly associated with the "it works on my machine" problem. Containerization provides an effective solution to this issue.

Containerization is a form of OS-level virtualization that allows applications to be deployed and executed in isolated environments without requiring a separate virtual machine for every application. Multiple containers can operate on the same host system while sharing the underlying operating system kernel.

Since containers share the host OS kernel, they are much more lightweight than traditional Virtual Machines. They can be started quickly and consume considerably fewer CPU, memory, and storage resources.

Within a DevOps workflow, containers are particularly useful for Continuous Integration and Continuous Deployment (CI/CD). They help maintain consistent and reproducible application environments throughout the different stages of the software development lifecycle.

2. What is Docker?

Docker is an open-source platform widely used for developing, deploying, scaling, and managing applications through containers. It provides the necessary tools and ecosystem to package an application along with all its required dependencies and run it consistently across different environments.

Core Docker Architecture and Components

- Docker Daemon (dockerd): A background service that runs on the host system and handles important Docker operations such as creating, running, and managing containers and images.
- Docker Client (docker): The command-line interface (CLI) through which users send commands and interact with the Docker Daemon.
- Docker Image: A read-only and immutable package that contains the application code, runtime environment, required libraries, dependencies, and other components needed to execute an application.
- Docker Container: A running instance of a Docker image. It provides an isolated environment in which the application is executed.
- Docker Registry (Docker Hub): A centralized repository used for storing, publishing, sharing, and downloading Docker images.

- Dockerfile: A text-based configuration file containing a sequence of instructions that Docker uses to automatically create a custom image.

3. Practical Execution and Command-Line Analysis

During the laboratory experiment, various Docker commands were executed to understand the different stages involved in managing containers. The purpose and functionality of each major operation are explained below.

3.1 Environment Verification and System Status

Before working with containers, it is important to confirm that Docker is correctly installed and operational.

- `docker --version`: This command displays the installed version of the Docker client. It can be used to verify the Docker installation and ensure that the required version is available. During the experiment, Docker version 29.2.1 was verified.
- `docker info`: This command displays detailed information about the Docker environment, including the number of containers, storage drivers, available resources, and other system-level configuration details.

3.2 Image Management and Retrieval

Docker images serve as the basic templates from which containers are created. Therefore, downloading, viewing, searching, and removing images are important Docker operations.

- `docker images`: This command displays the Docker images that are currently available on the local system. During the experiment, images such as `hello-world`, `myredis`, `redis`, and `welcome-to-docker` were present.
- `docker search <term>`: This command searches the public Docker Hub registry for images related to a particular keyword. For example, executing `docker search ubuntu` displays Ubuntu-related images along with information such as descriptions, star ratings, and whether an image is official.
- Applying Filters: Docker search results can be restricted using the `--filter` option. For example, `docker search --filter=stars=120 ubuntu` displays Ubuntu images having at least 120 community stars, making it easier to identify popular repositories.
- `docker pull <image_name>`: This command downloads the specified image from Docker Hub or another configured registry to the local system. During the experiment, the latest official Ubuntu image was successfully downloaded using `docker pull ubuntu`.
- `docker rmi <image_name>`: This command removes an image from the local Docker storage. An image generally cannot be deleted while it is being referenced by an existing container, including a stopped container, unless the `-f` option is used.
- `docker image prune`: This command removes dangling and unused image layers from the local system, helping to reclaim disk space.

3.3 Container Lifecycle Execution

The main stages of working with Docker involve creating, starting, monitoring, stopping, and removing containers.

- `docker run`: This is one of the most commonly used Docker commands. It creates a container from a specified image and starts it immediately.
- Interactive Execution (`-it`): The `-i` option keeps the input stream active, while `-t` provides a pseudo-terminal. Together, they allow the user to interact directly with a container's shell. For example, `docker run -it ubuntu /bin/bash` starts an Ubuntu container and opens its Bash shell. The container can be exited using the `exit` command.
- Detached Execution (`-d`): The `-d` option runs a container in the background without attaching it to the current terminal. This is useful for long-running services such as databases. For example, `docker run -d redis` starts a Redis container in detached mode.
- Port Mapping (`-p`): Port mapping allows applications running inside a container to be accessed from the host machine or external network. For example, `-p 6380:6379` maps port 6379 of the container to port 6380 on the host.
- Naming (`--name`): A custom name can be assigned to a container using the `--name` option. For example, `--name my_lab_container` makes it easier to identify and manage the container instead of relying on automatically generated names or IDs.
- `docker ps`: This command displays all currently running containers. Using `docker ps -a` or `docker ps --all` displays all containers, including those that have stopped or exited.
- `docker stop <container_name>`: This command stops a running container gracefully by sending a termination signal, allowing the application to shut down properly.
- `docker rm <container_name>`: This command removes a stopped container from the Docker environment.

3.4 Monitoring, Inspection, and Debugging

Docker includes several commands that help developers monitor container performance and identify problems. These tools are particularly useful in DevOps environments where quick troubleshooting is important.

- `docker container stats`: This command provides real-time information about the resource consumption of running containers, including CPU usage, memory consumption, network activity, and other performance statistics.
- `docker logs <container_name>`: This command displays the output and error messages generated by an application running inside a container. It is especially useful for troubleshooting applications running in detached mode.
- `docker inspect <container_name>`: This command provides detailed information about a container in JSON format. The output may include configuration details, network information, IP addresses, mounted volumes, and environment variables.
- `docker exec -it <container_name> /bin/bash`: This command allows the user to execute a command inside an already running container. In this case, it opens a Bash shell for interactive debugging. Unlike `docker run`, it does not create a new container.

3.5 Advanced Concepts: Image Building and Data Persistence

Docker also provides features for creating customized images and maintaining data even after containers are removed.

Building Custom Images

Name: Priyanshi Singh
Class: T22
Roll no: 2401115

- `docker build`: Custom Docker images can be created by defining the required environment and instructions inside a Dockerfile. The command `docker build -t <image_name> .` instructs Docker to read the Dockerfile from the current directory (`.`), build the image, and assign it the specified tag or name.

Data Volumes

By default, data stored inside a container is temporary. Therefore, when the container is deleted, the data stored within it may also be lost. This is not suitable for applications such as databases that require long-term data storage.

Docker Volumes provide a solution by offering persistent storage that exists independently of the container's lifecycle. A volume can be created using `docker volume create` and mounted into a container using the `-v` option. This ensures that important application or database data remains available even if the associated container is stopped or removed.

Name: Priyanshi Singh
Class: T22
Roll no: 2401115

Implementation :

```
Administrator: Command Pro
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\L15>docker --version
Docker version 29.2.1, build a5c7197

C:\Users\L15>docker images
                                                                    i Info →  U In Use
IMAGE                             ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest   c4d56c24da4f       22.2MB         6.03MB
hello-world:latest                ef54e839ef54       25.9kB         9.52kB    U
myredis:version1                  9199aba240ab        202MB         53.2MB
redis:latest                      009cc37796fb        204MB         55.3MB    U

C:\Users\L15>docker serach ubuntu
docker: unknown command: docker serach

Run 'docker --help' for more information

C:\Users\L15>docker search ubuntu
NAME                            DESCRIPTION                                     STARS     OFFICIAL
ubuntu                          Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid                    Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx                    Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9                    BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/kafka                    Apache Kafka, a distributed event streaming …   62
ubuntu/apache2                  Apache, a secure & extensible open-source HT…   104
ubuntu/zookeeper                ZooKeeper maintains configuration informatio…   15
ubuntu/prometheus               Prometheus is a systems and service monitori…   80
ubuntu/jre                      Chiseled Java runtime based on Ubuntu. Long-…   25
ubuntu/dotnet-aspnet            Chiselled Ubuntu runtime image for ASP.NET a…   26
ubuntu/python                   A chiselled Ubuntu rock with the Python runt…   33
ubuntu/mysql                    MySQL open source fast, stable, multi-thread…   74
ubuntu/postgres                 PostgreSQL is an open source object-relation…   45
ubuntu/redis                    Redis, an open source key-value store. Long-…   26
ubuntu/dotnet-runtime           Chiselled Ubuntu runtime image for .NET apps…   24
ubuntu/dotnet-deps              Chiselled Ubuntu for self-contained .NET & A…   16
ubuntu/grafana                  Grafana, a feature rich metrics dashboard & …   12
ubuntu/memcached                Memcached, in-memory keyvalue store for smal…   7
ubuntu/jdk                      Ubuntu rock for OpenJDK                         0
ubuntu/rabbitmq                 Ubuntu-based RabbitMQ image - an open source…   0
ubuntu/mimir                    Ubuntu ROCK for Mimir, a horizontally scalab…   0
ubuntu/loki                     Grafana Loki, a log aggregation system like …   2
ubuntu/traefik                  Ubuntu ROCK for Traefik, a modern HTTP rever…   3
ubuntu/opentelemetry-collector  This is a rock for the OpenTelemetry Collect…   0
ubuntu/alertmanager             Ubuntu Rock for Alertmanager. It handles ale…   1
```

```
C:\Users\L15>docker search --filter=stars=120 ubuntu
NAME           DESCRIPTION                                     STARS     OFFICIAL
ubuntu         Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid   Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx   Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9   BIND 9 is a very flexible, full-featured DNS…   120

C:\Users\L15>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
a3679419df18: Pull complete
ed819469700f: Pull complete
e16351a257e4: Download complete
Digest: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest

C:\Users\L15>docker images
                                                                    i Info →  U In Use
IMAGE                             ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest   c4d56c24da4f       22.2MB         6.03MB
hello-world:latest                ef54e839ef54       25.9kB         9.52kB    U
myredis:version1                  9199aba240ab        202MB         53.2MB
redis:latest                      009cc37796fb        204MB         55.3MB    U
ubuntu:latest                     3131b4cc82a7        161MB         45.3MB

C:\Users\L15>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\L15>docker run -it ubuntu /bin/bash
root@80c22874ea4d:/# exit
exit
```

Name: Priyanshi Singh
Class: T22
Roll no: 2401115

```
C:\Users\L15>docker info
Client:
 Version:    29.2.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: create or run AI agents (Docker Inc.)
    Version:  v1.27.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.18.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.31.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.0.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.3.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-desktop.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Program Files\Docker\cli-plugins\docker-extension.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.4.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-init.exe
  mcp: Docker MCP Plugin (Docker Inc.)
    Version:  v0.40.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-mcp.exe
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.1.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.5.56
    Path:     C:\Program Files\Docker\cli-plugins\docker-offload.exe
  pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
    Version:  v0.0.24
    Path:     C:\Program Files\Docker\cli-plugins\docker-pass.exe
  sandbox: Docker Sandbox (Docker Inc.)
    Version:  v0.12.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
  sbom: View the packaged-based Software Bill Of Materials (SBOM) for an image (Anchore Inc.)
    Version:  0.6.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-sbom.exe
  scout: Docker Scout (Docker Inc.)
    Version:  v1.20.0
```

```
Server:
 Containers: 4
  Running: 0
  Paused: 0
  Stopped: 4
 Images: 5
 Server Version: 29.2.1
 Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
 Logging Driver: json-file
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS


C:\Users\L15>docker run --name my_lab_redis -d redis
0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb

C:\Users\L15>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED                  STATUS         PORTS      NAMES
0b05c060a9f5   redis     "docker-entrypoint.s…"   Less than a second ago   Up 25 seconds   6379/tcp   my_lab_redis

C:\Users\L15>docker run --name my_mapped_redis -d -p 6380:6379 redis
0925a4ada1b2abb506f542f2334524d6f69ec6a399c9a5904e88854d4b67c2aba7

C:\Users\L15>docker logs my_lab_redis
Starting Redis Server
1:C 20 Jul 2026 10:22:12.909 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
1:C 20 Jul 2026 10:22:12.909 * Redis version=8.6.2, bits=64, commit=00000000, modified=1, pid=1, just started
1:C 20 Jul 2026 10:22:12.909 * Configuration loaded
1:M 20 Jul 2026 10:22:12.911 * monotonic clock: POSIX clock_gettime
1:M 20 Jul 2026 10:22:12.912 * Running mode=standalone, port=6379.
1:M 20 Jul 2026 10:22:12.914 * <bf> RedisBloom version 8.6.0 (Git=unknown)
1:M 20 Jul 2026 10:22:12.914 * <bf> Registering configuration options: [
1:M 20 Jul 2026 10:22:12.914 * <bf>     { bf-error-rate      :       0.01 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { bf-initial-size    :        100 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { bf-expansion-factor :         2 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-bucket-size     :         2 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-initial-size    :      1024 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-max-iterations  :        20 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-expansion-factor :         1 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-max-expansions  :        32 }
1:M 20 Jul 2026 10:22:12.914 * <bf> ]
1:M 20 Jul 2026 10:22:12.914 * Module 'bf' loaded from /usr/local/lib/redis/modules//redisbloom.so
1:M 20 Jul 2026 10:22:12.933 * <search> Redis version found by RedisSearch : 8.6.2 - oss
1:M 20 Jul 2026 10:22:12.933 * <search> RediSearch version 8.6.0 (Git=7782b97)
1:M 20 Jul 2026 10:22:12.933 * <search> Low level api version 1 initialized successfully
1:M 20 Jul 2026 10:22:12.933 * <search> gc: ON, prefix min length: 2, min word length to stem: 4, prefix max expansions: 200, query timeout (ms): 500, timeout policy: return, oom policy: return, cursor read size: 1000, cursor max idle (ms): 300000, max doctable size: 1000000, max number of search results:  1000000, default scorer: BM25STD,
1:M 20 Jul 2026 10:22:12.934 * <search> Initialized thread pools!
1:M 20 Jul 2026 10:22:12.934 * <search> Disabled workers threadpool of size 0
1:M 20 Jul 2026 10:22:12.934 * <search> Subscribe to config changes
1:M 20 Jul 2026 10:22:12.934 * <search> Subscribe to cluster slot migration events
```

Name: Priyanshi Singh
Class: T22
Roll no: 2401115

```
1:M 20 Jul 2026 10:22:12.934 * <search> Initialized thread pools!
1:M 20 Jul 2026 10:22:12.934 * <search> Disabled workers threadpool of size 0
1:M 20 Jul 2026 10:22:12.934 * <search> Subscribe to config changes
1:M 20 Jul 2026 10:22:12.934 * <search> Subscribe to cluster slot migration events
1:M 20 Jul 2026 10:22:12.934 * <search> Enabled role change notification
1:M 20 Jul 2026 10:22:12.935 * <search> Cluster configuration: AUTO partitions, type: 0, coordinator timeout: 0ms
1:M 20 Jul 2026 10:22:12.935 * Module 'search' loaded from /usr/local/lib/redis/modules//redisearch.so
1:M 20 Jul 2026 10:22:12.936 * <timeseries> RedisTimeSeries version 80600, git_sha=05fd355db74867686ldc4c17d19c8c1ca74c0154
1:M 20 Jul 2026 10:22:12.936 * <timeseries> Redis version found by RedisTimeSeries : 8.6.2 - oss
1:M 20 Jul 2026 10:22:12.936 * <timeseries> Registering configuration options: [
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-compaction-policy   :             }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-num-threads         :           3 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-retention-policy    :           0 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-duplicate-policy    :       block }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-chunk-size-bytes    :        4096 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-encoding            :  compressed }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-ignore-max-time-diff:           0 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-ignore-max-val-diff :    0.000000 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries> ]
1:M 20 Jul 2026 10:22:12.937 * <timeseries> Detected redis oss
1:M 20 Jul 2026 10:22:12.937 * <timeseries> Subscribe to ASM events
1:M 20 Jul 2026 10:22:12.937 * <timeseries> Enabled diskless replication
1:M 20 Jul 2026 10:22:12.937 * Module 'timeseries' loaded from /usr/local/lib/redis/modules//redistimeseries.so
1:M 20 Jul 2026 10:22:12.942 * <ReJSON> Created new data type 'ReJSON-RL'
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> version: 80600 git sha: unknown branch: unknown
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V1 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V2 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V3 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V4 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V5 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V6 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Enabled diskless replication
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Initialized shared string cache, thread safe: true.
1:M 20 Jul 2026 10:22:12.943 * Module 'ReJSON' loaded from /usr/local/lib/redis/modules//rejson.so
1:M 20 Jul 2026 10:22:12.943 * <search> Acquired RedisJSON_V6 API
1:M 20 Jul 2026 10:22:12.944 * Server initialized
1:M 20 Jul 2026 10:22:12.944 * Ready to accept connections tcp
1:M 20 Jul 2026 10:22:12.944 # WARNING: Redis does not require authentication and is not protected by network restrictions. Redis will accept connections from any IP address on any network interface.

C:\Users\L15>docker inspect my_lab_redis
[
    {
        "Id": "0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb",
        "Created": "2026-07-20T10:22:12.673590477Z",
        "Path": "docker-entrypoint.sh",
        "Args": [
            "redis-server"
        ],
```

```
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 542,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-20T10:22:12.718942097Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
        "Image": "sha256:009cc37796fbdbe1b631b4cc0582bed167e5e403ed8bcd06f77eb6cb5aeb6f93",
        "ResolvConfPath": "/var/lib/docker/containers/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb/hostname",
        "HostsPath": "/var/lib/docker/containers/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb/hosts",
        "LogPath": "/var/lib/docker/containers/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb-json.log",
        "Name": "/my_lab_redis",
        "RestartCount": 0,
        "Driver": "overlayfs",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
                "Config": {}
            },
            "NetworkMode": "bridge",
            "PortBindings": {},
            "RestartPolicy": {
                "Name": "no",
                "MaximumRetryCount": 0
            },
            "AutoRemove": false,
            "VolumeDriver": "",
            "VolumesFrom": null,
            "ConsoleSize": [
                51,
                138
            ],
            "CapAdd": null,
            "CapDrop": null,
            "CgroupnsMode": "private",
            "Dns": null,
```

Name: Priyanshi Singh
Class: T22
Roll no: 2401115


```
            "CgroupnsMode": "private",
            "Dns": null,
            "DnsOptions": [],
            "DnsSearch": [],
            "ExtraHosts": null,
            "GroupAdd": null,
            "IpcMode": "private",
            "Cgroup": "",
            "Links": null,
            "OomScoreAdj": 0,
            "PidMode": "",
            "Privileged": false,
            "PublishAllPorts": false,
            "ReadonlyRootfs": false,
            "SecurityOpt": null,
            "UTSMode": "",
            "UsernsMode": "",
            "ShmSize": 67108864,
            "Runtime": "runc",
            "Isolation": "",
            "CpuShares": 0,
            "Memory": 0,
            "NanoCpus": 0,
            "CgroupParent": "",
            "BlkioWeight": 0,
            "BlkioWeightDevice": [],
            "BlkioDeviceReadBps": [],
            "BlkioDeviceWriteBps": [],
            "BlkioDeviceReadIOps": [],
            "BlkioDeviceWriteIOps": [],
            "CpuPeriod": 0,
            "CpuQuota": 0,
            "CpuRealtimePeriod": 0,
            "CpuRealtimeRuntime": 0,
            "CpusetCpus": "",
            "CpusetMems": "",
            "Devices": [],
            "DeviceCgroupRules": null,
            "DeviceRequests": null,
            "MemoryReservation": 0,
            "MemorySwap": 0,
            "MemorySwappiness": null,
            "OomKillDisable": null,
            "PidsLimit": null,
            "Ulimits": [],
            "CpuCount": 0,
            "CpuPercent": 0,
            "IOMaximumIOps": 0,
            "IOMaximumBandwidth": 0,
            "MaskedPaths": [
```

Name: Priyanshi Singh
Class: T22
Roll no: 2401115


```
            "MaskedPaths": [
                "/proc/acpi",
                "/proc/asound",
                "/proc/interrupts",
                "/proc/kcore",
                "/proc/keys",
                "/proc/latency_stats",
                "/proc/sched_debug",
                "/proc/scsi",
                "/proc/timer_list",
                "/proc/timer_stats",
                "/sys/devices/virtual/powercap",
                "/sys/firmware"
            ],
            "ReadonlyPaths": [
                "/proc/bus",
                "/proc/fs",
                "/proc/irq",
                "/proc/sys",
                "/proc/sysrq-trigger"
            ]
        },
        "Storage": {
            "RootFS": {
                "Snapshot": {
                    "Name": "overlayfs"
                }
            }
        },
        "Mounts": [],
        "Config": {
            "Hostname": "0b05c060a9f5",
            "Domainname": "",
            "User": "",
            "AttachStdin": false,
            "AttachStdout": false,
            "AttachStderr": false,
            "ExposedPorts": {
                "6379/tcp": {}
            },
            "Tty": false,
            "OpenStdin": false,
            "StdinOnce": false,
            "Env": [
                "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
            ],
            "Cmd": [
                "redis-server"
            ],
            "Image": "redis",
```

Name: Priyanshi Singh
Class: T22
Roll no: 2401115


```
      "Image": "redis",
      "Volumes": null,
      "WorkingDir": "/data",
      "Entrypoint": [
          "docker-entrypoint.sh"
      ],
      "Labels": {},
      "StopTimeout": 1
  },
  "NetworkSettings": {
      "SandboxID": "283db676f7387b958d610f44f991278a3d428d04a916ef33010047d52d7c9f99",
      "SandboxKey": "/var/run/docker/netns/283db676f738",
      "Ports": {
          "6379/tcp": null
      },
      "Networks": {
          "bridge": {
              "IPAMConfig": null,
              "Links": null,
              "Aliases": null,
              "DriverOpts": null,
              "GwPriority": 0,
              "NetworkID": "ca0543cf5dac9e6195a9e279fdb5a4048957b9ccc1aff3c976da008e7400abdf",
              "EndpointID": "bdd3b732a79179852c71990f2b342e3d9ac4d592ed19c1096d65950a3d61b615",
              "Gateway": "172.17.0.1",
              "IPAddress": "172.17.0.2",
              "MacAddress": "c6:85:95:c8:d8:a9",
              "IPPrefixLen": 16,
              "IPv6Gateway": "",
              "GlobalIPv6Address": "",
              "GlobalIPv6PrefixLen": 0,
              "DNSNames": null
          }
      }
  },
  "ImageManifestDescriptor": {
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "digest": "sha256:17a235ee842cd21d8bcb7f11a49fb177cbeaf5537996304a0089e94412de2ed5",
      "size": 2286,
      "annotations": {
          "com.docker.official-images.bashbrew.arch": "amd64",
          "org.opencontainers.image.base.digest": "sha256:edc9450a9fe37d30b508808052f8d0564e3ed9eaf565e043c6f5632957f7381e",
          "org.opencontainers.image.base.name": "debian:trixie-slim",
          "org.opencontainers.image.created": "2026-03-24T18:12:25Z",
          "org.opencontainers.image.revision": "8c81e2a44cae9258294718d767ce594e5cbbf20e",
          "org.opencontainers.image.source": "https://github.com/redis/docker-library-redis.git#8c81e2a44cae9258294718d767ce594e5cbbf20e:debian",
          "org.opencontainers.image.url": "https://hub.docker.com/_/redis",
          "org.opencontainers.image.version": "8.6.2"
      },
      "platform": {
          "architecture": "amd64",
```

```
          "platform": {
              "architecture": "amd64",
              "os": "linux"
          }
      }
  }
]

C:\Users\L15>docker stop my_lab_redis
my_lab_redis

C:\Users\L15>docker stop my_mapped_redis
my_mapped_redis

C:\Users\L15>docker ps --all
CONTAINER ID   IMAGE         COMMAND                  CREATED                  STATUS                      PORTS     NAMES
0925a4ada1b2   redis         "docker-entrypoint.s…"   Less than a second ago   Exited (0) 8 seconds ago              my_mapped_redis
0b05c060a9f5   redis         "docker-entrypoint.s…"   Less than a second ago   Exited (0) 16 seconds ago             my_lab_redis
80c22874ea4d   ubuntu        "/bin/bash"              Less than a second ago   Exited (0) 4 minutes ago              relaxed_almeida
83ab4fe6a825   redis         "docker-entrypoint.s…"   3 months ago             Exited (0) 3 months ago               crazy_agnesi
b7a18b45f217   redis         "docker-entrypoint.s…"   3 months ago             Exited (0) 3 months ago               awesome_allen
222596f84fc0   hello-world   "/hello"                 4 months ago             Exited (0) 4 months ago               gracious_wilson

C:\Users\L15>docker rm my_lab_redis
my_lab_redis

C:\Users\L15>docker rm my_mapped_redis
my_mapped_redis

C:\Users\L15>docker rmi hello-world
Error response from daemon: conflict: unable to delete hello-world:latest (must be forced) - container 222596f84fc0 is using its referenced image ef54e839ef54

C:\Users\L15>docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B

C:\Users\L15>docker rm 222596f84fc0
222596f84fc0

C:\Users\L15>docker rmi hello-world
Untagged: hello-world:latest
Deleted: sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a

C:\Users\L15>echo FROM ubuntu > Dockerfile

C:\Users\L15>echo CMD ["echo", "My custom image build was successful!"] >> Dockerfile

C:\Users\L15>docker build -t my_custom_image .
[+] Building 0.3s (5/5) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                              0.0s
```

Name: Priyanshi Singh
Class: T22
Roll no: 2401115

```
C:\Users\L15>docker build -t my_custom_image .
[+] Building 0.3s (5/5) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 107B                                                                                                0.0s
 => [internal] load metadata for docker.io/library/ubuntu:latest                                                                    0.0s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 2B                                                                                                     0.0s
 => [1/1] FROM docker.io/library/ubuntu:latest@sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb              0.0s
 => => resolve docker.io/library/ubuntu:latest@sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb              0.0s
 => exporting to image                                                                                                              0.1s
 => => exporting layers                                                                                                             0.0s
 => => exporting manifest sha256:165c2f52bc45dd0b98877fc9437e2833db17f21f9d80b3c7254f94aca5eccb60                                   0.0s
 => => exporting config sha256:f36bef7221b0c2d4123a6fdfdbd626e405ff1b44e13457569a1e5e87b6d0a85f                                     0.0s
 => => exporting attestation manifest sha256:7227e9785405e65c044d93fbae0d3f380931ea1ce7c459300f9bec5251ce26bc                       0.0s
 => => exporting manifest list sha256:76ac716dc74323242ee0613826fcaff286c99740760323c0cedcf6e243623269                              0.0s
 => => naming to docker.io/library/my_custom_image:latest                                                                           0.0s
 => => unpacking to docker.io/library/my_custom_image:latest                                                                        0.0s

C:\Users\L15>docker run my_custom_image
My custom image build was successful!

C:\Users\L15>docker run -d --name live_test_container ubuntu sleep 3600
7e6d5b1ab280e7d3b6270f4605fb0159b24daf16980feb5b7af71173b86f81ca

C:\Users\L15>docker exec -it live_test_container /bin/bash
root@7e6d5b1ab280:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@7e6d5b1ab280:/# pwd
/
root@7e6d5b1ab280:/# mkdir trial
root@7e6d5b1ab280:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  trial  usr  var
root@7e6d5b1ab280:/# cd trial
root@7e6d5b1ab280:/trial# exit
exit

C:\Users\L15>docker volume create my_database_data
my_database_data

C:\Users\L15>docker run -d --name my_mysql -v my_database_data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=root mysql
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
81b3a6a3bce8: Pull complete
e0607e526570: Pull complete
5ac73e0e3f6a: Pull complete
bb49c3208fe7: Pull complete
61d49498d3da: Pull complete
c2fb254fc2f4: Pull complete
08b808cb6d40: Pull complete
1576763b528a: Pull complete
e21df1635f12: Pull complete
```

```
C:\Users\L15>docker run -d --name my_mysql -v my_database_data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=root mysql
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
81b3a6a3bce8: Pull complete
e0607e526570: Pull complete
5ac73e0e3f6a: Pull complete
bb49c3208fe7: Pull complete
61d49498d3da: Pull complete
c2fb254fc2f4: Pull complete
08b808cb6d40: Pull complete
1576763b528a: Pull complete
e21df1635f12: Pull complete
88745f4755f0: Pull complete
a701ef10ab46: Download complete
1f9f19898688: Download complete
Digest: sha256:8fdf311514c91fa5014e93e98e19d7f5d9eb162a462c13100c956dacc278ee21
Status: Downloaded newer image for mysql:latest
97a63a9a708641ad3e6de14efb10431d8cd84f05b5a4765ea105884c80e12f9a

C:\Users\L15>docker stop live_test_container my_mysql
live_test_container
my_mysql

C:\Users\L15>docker rm live_test_container my_mysql
live_test_container
my_mysql

C:\Users\L15>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\L15>
```

Conclusion :

Name: Priyanshi Singh
Class: T22
Roll no: 2401115

In conclusion, this experiment successfully demonstrated the practical application of containerization and environment isolation using the Docker framework. By managing official image repositories, executing interactive and detached container lifecycles, and establishing host-port mapping, the core competencies of consistent application deployment outlined in Learning Outcome 4 (LO4) were achieved. Furthermore, simulating container resource monitoring and system inspections underscored the importance of environment reproducibility and infrastructure immutability within collaborative Agile and DevOps engineering environments.

LO4 mapped successfully.